

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 02 -

SHIFT-LEFT: SAST, DAST E SCA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	10
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

O QUE VEM POR AÍ?

A segurança é o novo foco central do ciclo de desenvolvimento, e a proteção das credenciais digitais que interligam suas aplicações à *cloud* é o risco mais crítico. Uma falha aqui resulta no *Secret Sprawl*, a proliferação descontrolada de chaves de acesso no código, transformando o controle de versão em um ponto de vulnerabilidade.

Nesta aula, vamos neutralizar essa ameaça com estratégias práticas e ferramentas técnicas. O objetivo é implementar varreduras de segredos na linha de frente e arquitetar soluções que garantam que as credenciais de produção sejam efêmeras. Prepare-se para integrar segurança *zero-trust* diretamente no seu código e nos seus *pipelines*.

HANDS ON

A teoria de segurança arquitetural e a governança de código se materializam em uma jornada prática de defesa em profundidade, cobrindo o ciclo de vida da credencial, desde o desenvolvimento local até a execução em tempo real. O foco inicial é a prevenção imediata, onde você será guiado na configuração do Gitleaks no *pipeline* de Integração Contínua (CI). Esta etapa é crucial para bloquear qualquer *commit* que tente introduzir chaves de acesso AWS ou tokens sensíveis, garantindo que o código contaminado jamais chegue ao repositório central.

Em seguida, a aula complementar essa defesa, ensinando como institucionalizar a segurança no repositório por meio da configuração de *Branch Protection Rules* e *Pre-Commit Hooks*. Essa camada de governança assegura que a integridade do código seja um pré-requisito técnico obrigatório, impedindo a mesclagem de código sem o sucesso das varreduras e *status checks*.

Por fim, a solução arquitetural de *runtime* será explorada com foco na gestão dinâmica de segredos. A prática demonstrará a substituição de variáveis de ambiente estáticas pela integração com o AWS Secrets Manager. O objetivo é que a aplicação busque a credencial em tempo de execução, resolvendo o risco inerente ao armazenamento de segredos de produção no *pipeline* de CI/CD ou no código fonte.

SAIBA MAIS

O Cenário de Risco e o Secret Sprawl na Era DevSecOps

No desenvolvimento *cloud-native*, credenciais digitais (chaves de API, senhas, tokens) são vitais para a comunicação entre sistemas. O *Secret Sprawl* ocorre quando essas credenciais se dispersam e são armazenadas indevidamente em arquivos de configuração, *logs* ou, de forma mais crítica, no histórico de *commits* do Git. Essa falha é uma das maiores fontes de risco em TI, com milhões de novos segredos expostos anualmente em plataformas de código. A transição para DevSecOps é a resposta, integrando segurança de forma proativa para mitigar o risco antes que o segredo alcance o repositório central.

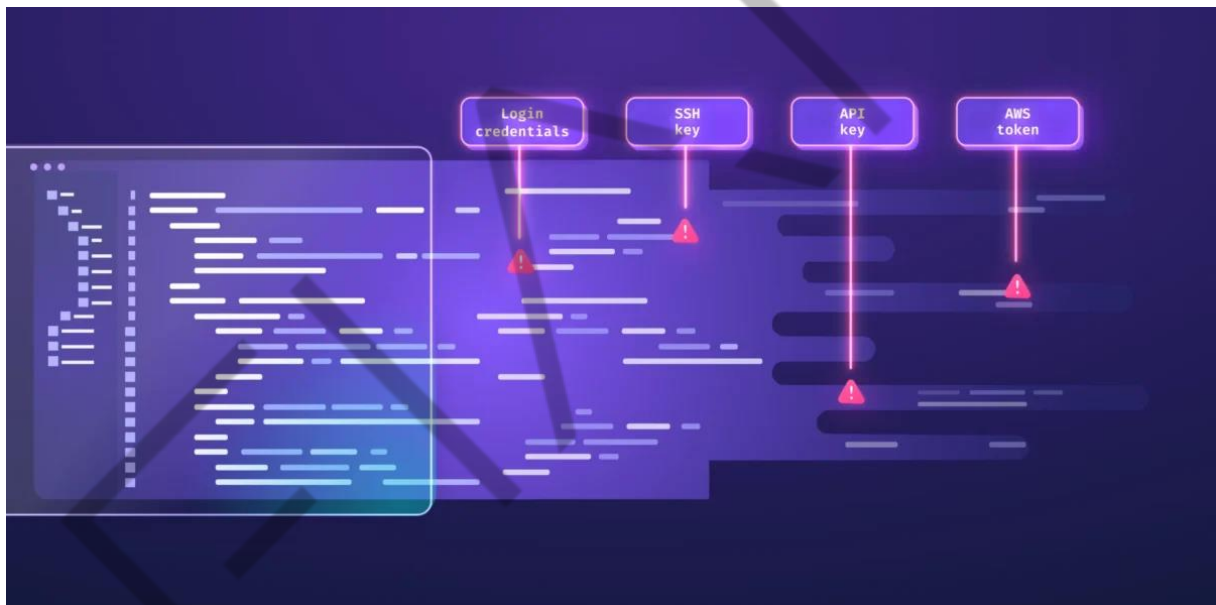


Figura 1 – Secret Sprawl
Fonte: [Checkmarx](#) (2025)

Estratégias de Defesa em Profundidade no Git

Combater o *Secret Sprawl* exige defesa em camadas, começando pelo desenvolvedor.

Bloqueio Proativo: A Implementação de Pre-Commit Hooks

A primeira linha de defesa é o *pre-commit hook*, um *script* do Git que executa localmente antes da conclusão do *commit*. Ele inspeciona o código para impedir a

entrada de segredos sensíveis no histórico do Git. O Gitleaks, ferramenta de código aberto, é ideal para essa varredura, utilizando expressões regulares e análise de alta entropia. Ao ser configurado como *pre-commit hook* (preferencialmente via um gerenciador como *pre-commit framework*), ele bloqueia automaticamente *commits* que contenham segredos. Essa estratégia *shift left* otimiza processos, pois corrigir o vazamento localmente é imediato e de baixo custo.

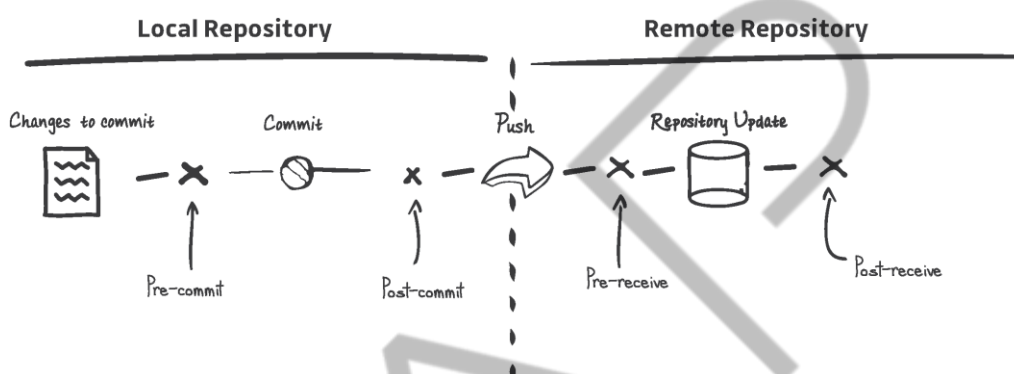


Figura 2 – Pre-Commit Hooks
Fonte: [Gitguardian](#) (2025)

Varredura Contínua (Secret Scanning) no Pipeline

Como os *pre-commit hooks* podem ser ignorados, a varredura contínua no pipeline de CI/CD serve como uma rede de segurança essencial. Integrada como um *status check* obrigatório para Pull Requests (PRs), ferramentas como o Gitleaks são executadas no código enviado. Se um segredo for detectado, o *build* falha e a fusão do código na *branch* principal é bloqueada, garantindo conformidade e capturando falhas na prevenção local.

Proteção da Integridade do Repositório Central: Branch Protection Rules

As *Branch Protection Rules* são mecanismos críticos para governar a mesclagem de código em *branches* de missão crítica. Elas atuam como *gatekeepers* de segurança no fluxo DevSecOps. Três requisitos técnicos vitais são: **Require Status Checks before Merging** (exige que varreduras de segurança, como o *Secret Scanning*, sejam aprovadas), **Require Signed Commits** (garante a autenticidade criptográfica do autor, vital para rastreabilidade) e **Require Linear History** (impede a reescrita do histórico do Git, mantendo uma trilha de auditoria limpa).

Gestão de Identidade, Acesso e o Ciclo de Vida de Credenciais

Princípios de IAM e Boas Práticas

A gestão segura de acesso deve seguir o princípio de **Minimização de Risco**, priorizando o uso de credenciais temporárias (via IAM Roles ou federação) e a imposição de Multi-Factor Authentication (MFA). O Princípio do Menor Privilégio (*Least Privilege*) é fundamental, garantindo que usuários e serviços tenham apenas as permissões estritamente necessárias. A meta arquitetural é eliminar a dependência de credenciais de longa duração armazenadas fora de um cofre gerenciado.

Diferenciação Crucial: Segredos de Build vs. Segredos de Runtime

É crucial separar credenciais:

- Segredos de Build (CI Secrets):** usados pelo ambiente de CI (ex: tokens para Docker Registry). Devem ser armazenados nos cofres nativos do CI/CD (ex: GitHub Actions Secrets).
- Segredos de Runtime (Application Secrets):** usados pela aplicação em execução (ex: credenciais de banco de dados de produção). Devem ser gerenciados por cofres de nuvem especializados (ex: AWS Secrets Manager).

Armazenar Segredos de Runtime no CI/CD é um erro grave. O modelo ideal é que a aplicação em *runtime* busque a credencial dinamicamente, mantendo o *pipeline* com o menor privilégio.

Tipo de Segredo	Consumidor	Fase do Ciclo	Local de Armazenamento Recomendado
Segredos de Build (CI)	Pipeline de CI/CD (Jobs).	Integração Contínua (Build, Teste, Deploy).	Cofres Nativos do CI/CD (GitHub Secrets, GitLab Variables).
Segredos de Runtime (App)	Aplicação em Execução (Containers, Serverless)	Entrega Contínua (Runtime).	Cofres de Nuvem Gerenciados (AWS Secrets Manager, Azure Key Vault).

Tabela 1 – Diferenciação de Segredos no Ciclo DevSecOps

 Fonte: Elaborado pelo autor (2025)

AWS Secrets Manager: Arquitetura e Consumo em Runtime

O AWS Secrets Manager (ASM) é um serviço gerenciado para armazenar, recuperar e rotacionar credenciais. Ele substitui credenciais *hard-coded* por uma chamada de API dinâmica. O fluxo é seguro: o segredo é armazenado criptografado (via KMS) e o serviço consumidor autentica-se usando um IAM Role (com permissão `secretsmanager:GetSecretValue`) para obter o valor. A credencial é entregue apenas em memória, o que impede a inspeção e comprometimento por terceiros.

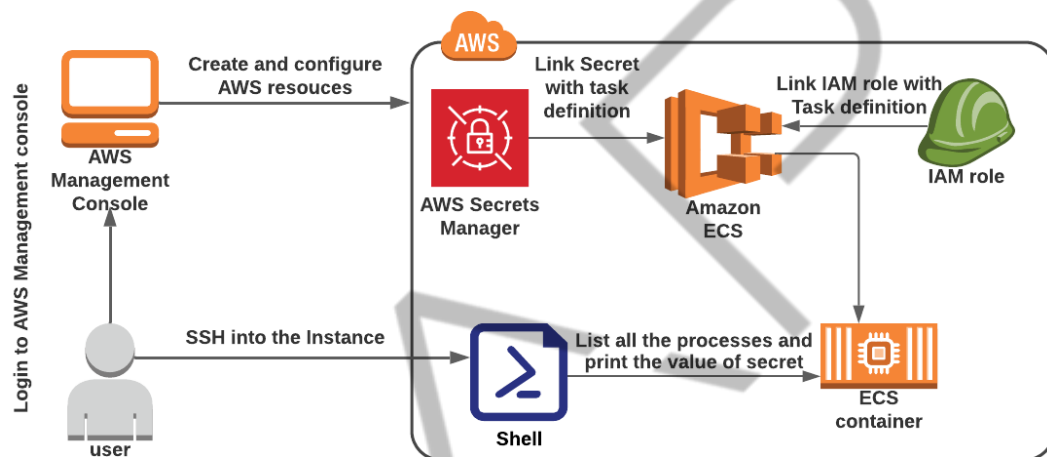


Figura 3 – AWS Secret Manager
Fonte: [Labresources](#) (2025)

Rotação Automatizada de Chaves com AWS Lambda

A rotação automatizada transforma credenciais em efêmeras, limitando o tempo de exposição. Para muitos segredos, o Secrets Manager utiliza uma função AWS Lambda customizável para orquestrar a rotação em um cronograma. Para garantir uma transição **zero-downtime**, o serviço usa *staging labels*: `AWSCURRENT` (versão em uso) e `AWSPENDING` (nova versão em teste). O processo de rotação da função Lambda segue quatro etapas rigorosas (Criação, Definição do `AWSPENDING`, Teste e Promoção para `AWSCURRENT`), assegurando que a aplicação utilize sempre uma credencial válida.

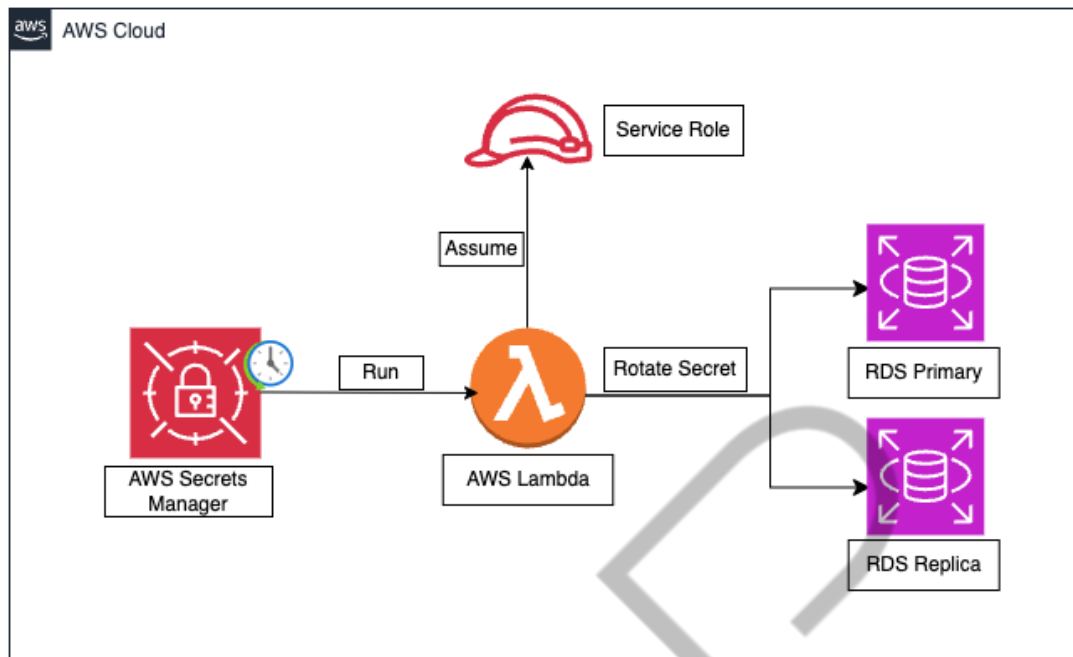


Figura 4 – Rotação AWS Secret Manager
Fonte: [AWS](#) (2025)

MERCADO, CASES E TENDÊNCIAS

A gestão proativa de segredos é um imperativo estratégico impulsionado pela complexidade das arquiteturas *cloud-native* e pela alta taxa de exposição de credenciais. O mercado está respondendo à crise sistêmica onde milhões de credenciais continuam sendo expostas em plataformas de código anualmente, muitas delas permanecendo ativas e exploráveis.

Casos de comprometimento demonstram que a exploração de uma única chave de API vazada, mesmo que acidentalmente, pode contornar arquiteturas de segurança com investimentos significativos. Essa realidade forçou grandes *players*, como GitHub e AWS, a investirem pesadamente em mecanismos de detecção proativa e revogação automática de chaves, validando a importância crítica da varredura contínua e da rotação automatizada para limitar o tempo de exposição.

Uma tendência clara de DevSecOps é o reconhecimento de que o *Secret Sprawl* não se restringe apenas ao código fonte. A análise de mercado revela que ferramentas de colaboração amplamente utilizadas, como Slack e Jira, tornaram-se vetores significativos de vazamento de credenciais. Nesses ambientes, a gravidade dos incidentes é frequentemente classificada como crítica, exigindo a expansão das ferramentas de *Secret Scanning* para escanear todo o ecossistema digital da empresa.

Paralelamente, a gestão de segredos está se consolidando em torno de plataformas de cofres gerenciados, como o AWS Secrets Manager ou o HashiCorp Vault. Essa centralização é crucial para padronização, auditoria de acesso e conformidade regulatória. Essa infraestrutura é um dos pilares da crescente disciplina de Engenharia de Plataformas.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, a segurança no controle de versão foi abordada de forma estratégica, focando na prevenção e na arquitetura de gestão de segredos. Você compreendeu o risco do *Secret Sprawl* e implementou camadas de defesa, começando pela varredura proativa com Gitleaks em *pre-commit hooks* e a imposição de Branch Protection Rules para garantir a integridade do repositório antes da mesclagem.

O ponto arquitetural mais crítico foi a clara distinção entre Segredos de Build e Segredos de Runtime, pavimentando o caminho para a integração segura com o AWS Secrets Manager. Finalmente, você dominou o conceito de rotação automatizada via AWS Lambda e o uso de *Staging Labels* para assegurar que as credenciais de *runtime* sejam efêmeras, limitando o risco de comprometimento sem causar indisponibilidade da aplicação.

REFERÊNCIAS

AWS. **AWS Identity and Access Management best practices**. 2025. Disponível em: <https://aws.amazon.com/iam/resources/best-practices/>. Acesso em: 28 nov. 2025.

AWS. **Four steps in a rotation function**. 2025. Disponível em: https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotate-secrets_lambda-functions.html. Acesso em: 28 nov. 2025.

AWS. **How AWS Secrets Manager works**. 2025. Disponível em: <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>. Acesso em: 28 nov. 2025.

DEVSECOPS SCHOOL. **Secrets management in DevSecOps: a comprehensive guide**. 2025. Disponível em: <https://devsecopsschool.com/blog/secrets-management-in-devsecops-a-comprehensive-guide/>. Acesso em: 28 nov. 2025.

GITGUARDIAN. **The State of Secrets Sprawl 2025: A worsening crisis**. 2025. Disponível em: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2025>. Acesso em: 28 nov. 2025.

GITHUB. **About protected branches**. 2025. Disponível em: <https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>. Acesso em: 28 nov. 2025.

GITLEAKS. **Gitleaks: Scan git repos for secrets**. 2025. Disponível em: <https://gitleaks.io/>. Acesso em: 28 nov. 2025.

PALAVRAS-CHAVE

DevSecOps. Secret Scanning. Gitleaks. AWS Secrets Manager. AWS Lambda.
Branch Protection. Credenciais Temporárias. Runtime Secrets. IAM. Secret Sprawl.
Pre-commit Hook. Staging Labels.

EMSE



POS TECH